

© Joshua M. Willman 2020

J. M. Willman, *Beginning PyQt*

[https://doi-org.ezproxy.sfpl.org/10.1007/978-1-4842-5857-6\\_6](https://doi-org.ezproxy.sfpl.org/10.1007/978-1-4842-5857-6_6)

## 6. Styling Your GUIs

Joshua M. Willman<sup>1</sup>

(1) Hampton, VA, USA

The GUIs you have created up until now have all focused mainly on function and less on appearance and customization. Creating a layout to organize widgets in a coherent manner is just as important as modifying the look and feel of each and every widget. By choosing the right styles, colors, and fonts, a user can also more easily navigate their way around a user interface.

In this chapter, we will be taking a look at why customizing the look of widgets, windows, and actions is also necessary for designing great GUIs.

In the final section of the chapter, we will take another look at event handling in PyQt and see how we can modify signals and slots to create custom signals to further improve the potential of applications.

Chapter 6 illustrates how to

- Modify the appearance of widgets with `Qt Style Sheets`
- Utilize new Qt widgets and classes, including `QRadioButton`, `QGroupBox`, and `QTabWidget`
- Reimplement event handlers
- Create custom signals using `pyqtSignal` and `QObject`

### Changing GUI Appearances with Qt Style Sheets

When you use PyQt, the appearance of your applications is handled by Qt's **QStyle** class. QStyle contains a number of subclasses that imitate the look of the system on which the application is being run. This makes your GUI look like a native MacOS, Linux, or Windows application. Custom styles can be made either by modifying existing QStyle classes, creating new classes, or using Qt Style Sheets.

This chapter will take a look at how to create custom styles by using style sheets. **Qt Style Sheets** provide a technique for customizing the look of widgets. The syntax used in Qt Style Sheets is inspired by HTML Cascading Style Sheets (CSS).

With style sheets, you can customize the look of a number of different widget properties, pseudostates, and subcontrols. Some of the properties that you can modify include background color, font size and font color, border type, width or style, as well as add padding to widgets.

**Pseudostates** are used to define special states of a widget, such as when a

mouse hovers over a widget or when a widget changes states from active to disabled. **Subcontrols** allow you to access a widget's subelements and change their appearance, location, or other properties. For example, you could change the indicator of a `QCheckBox` to a different color when it is checked or unchecked.

Customizations can either be applied to individual widgets or to the `QApplication` object by using `setStyleSheet()`.

### Customizing Individual Widget Properties

Let's start by seeing how to apply changes to widgets. The following code changes the background color to blue:

```
line_edit.setStyleSheet("background-color: blue")
```

Colors in a style sheet can be specified using either hexadecimal, RGB, or color keyword formats. To change the foreground color (the text color) of a widget

```
line_edit.setStyleSheet("color: rgb(244, 160, 25") # orange
```

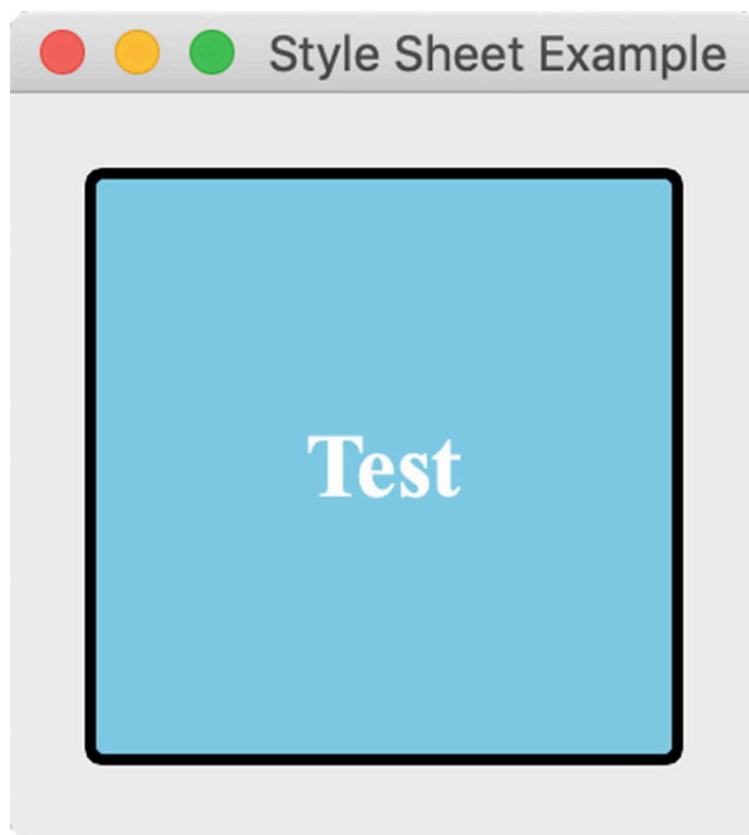
For some widgets as well the main window, you can even set a background image.

```
self.setStyleSheet("background-image: url(images/logo.png)")
```

Now let's take a look at more a detailed example. For the following `QLabel` widget, we will see how to change the color, the border, some font properties, and the text alignment. The results are shown in Figure [6-1](#).

```
label = QLabel("Test", self)
label.setStyleSheet("""background-color: skyblue;
    color: white;
    border-style: outset;
    border-width: 3px;
    border-radius: 5px;
    font: bold 24px 'Times New Roman';
    qproperty-alignment: AlignCenter""")
```

Of course, this is but one example. Each of the different kinds of widgets in Qt has its own parameters that can be customized. For a list of properties that are supported by Qt Style Sheets, refer to Appendix A.



**Figure 6-1** A customized QLabel widget with sky blue background and rounded corners

## Customizing the QApplication Style Sheet

If you have multiple widgets of the same type in an application, you could set each individual widget's style one by one. However, if those widgets all have the same properties, then a much simpler method is to specify all of the modifications at one time.

```
app = QApplication(sys.argv)
app.setStyleSheet("QPushButton{background-color: #C92108}")
```

This will apply a red color to all QPushButton widgets in the GUI. However, if you only want the properties to apply to a specific QPushButton, you can give it an ID selector using `setObjectName()`. The following excerpt of code shows how to use the ID selector to refer to a particular button. When the button is pressed, a different background color is used.

```
style_sheet = """
    QPushButton#Warning_Button{
        background-color: #C92108;
        border-radius: 10px;
        padding: 6px;
        color: #FFFFFF
    }
    QPushButton#Warning_Button:pressed{
        background-color: #F4B519;
    }
    """

button = QPushButton("Warning!", self)
button.setObjectName("Warning_Button") # Set ID selector
app = QApplication(sys.argv)
```

```
app.setStyleSheet(style_sheet) # Set style of QApplication
```

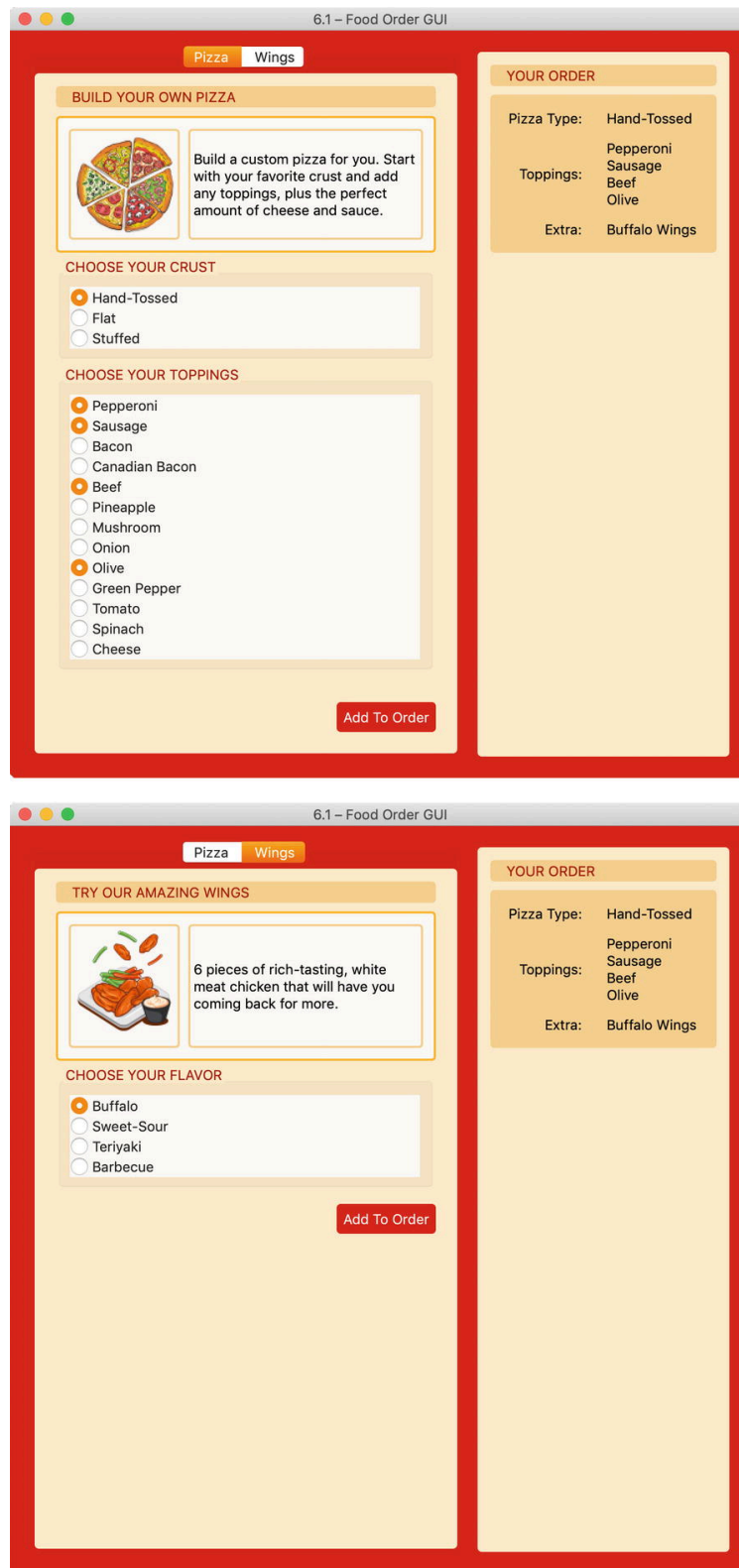
The preceding code also demonstrates how to create a `style_sheet` variable that contains the different properties for each widget. To add a different type of class, simply include the widget type such as `QCheckBox` followed by the attributes to be changed.

## Project 6.1 – Food Ordering GUI

Food delivery service apps are everywhere – on your phone, on the Internet, and even on kiosks when you go to actual restaurants themselves. They simplify the ordering process while also giving the user a feeling of control over their choices, asking us to select our own foods and items as we scroll through a list of organized categories.

These types of GUIs may possibly need to contain hundreds of different items that fit into multiple groups. Rather than just throwing all of the products into the interface and letting the user waste their own time sorting through the items, goods are usually placed into categories often differentiated by tabs. These tabs contain titles for the products that can be found on those corresponding pages, such as Frozen Foods or Fruits/Vegetables.

The GUI in this project allows the user to place an order for a pizza. It lays a foundation for a food ordering application, using tab widgets to organize items onto separate pages. The project also shows how you can use style sheets to give a GUI made using PyQt5 a more visually pleasing appearance. The application can be seen in Figure [6-2](#).



**Figure 6-2** The food ordering GUI. The GUI contains two tabs, Pizza (top) and Wings (bottom), to separate the types of food a customer can see at one time. The choices that can be selected, which are `QRadioButton` widgets, are separated using `QGroupBox` widgets. The main window has a red background, and each tab has a tan background. These colors and other styles are created by using a style sheet

### Design the Food Ordering GUI

This application consists of two main tabs (displayed in Figure 6-3), but more could be easily added. Each tab consists of a `QWidget` that acts as a container for all of the other widgets. The first tab, Pizza, contains an image and text to convey the purpose of the tab to the user. This is followed

by two `QGroupBox` widgets that each consist of a number of `QRadioButton` widgets. While the radio buttons in the “Crust” group box are mutually exclusive, the ones in the “Toppings” group box are not, so that the user can select multiple options at one time.

The second tab, Wings, is set up in a similar fashion with the “Flavor” radio buttons being mutually exclusive.

At the bottom of each page is an “Add to Order” `QPushButton` that will update the user’s order in the widget on the right-hand side of the window.

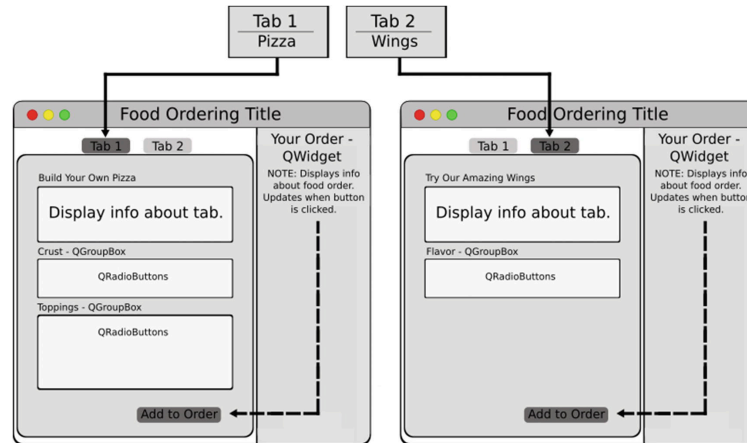


Figure 6-3 The design for the food ordering GUI

Before we look at the code for the food ordering GUI, let’s take a moment to learn about the new Qt classes in this project – `QGroupBox`, `QRadioButton`, and `QTabWidget`.

### The `QRadioButton` Widget

The `QRadioButton` class allows you to create option buttons that can be switched on when checked or off when unchecked. Each radio button consists of a round button and a corresponding label or icon. Radio buttons are generally used for situations where you need to provide a user with multiple choices, but only one choice can be checked at a time. As the user selects a new radio button, the other radio buttons are unchecked.

When you place multiple radio buttons in a parent widget, those buttons become **auto-exclusive**, meaning they automatically become exclusive members of that group. If one radio button is checked inside of the parent, all of the other buttons will become unchecked. To change this functionality, you can set the `setAutoExclusive()` attribute to `False`.

Also, if you want to place multiple exclusive groups of radio buttons into the same parent widget, then use the `QButtonGroup` class to keep the different groups separate. Refer back to Chapter 4 for information about `QButtonGroup`.

Radio buttons are similar to the `QCheckBox` class when emitting signals. A radio button emits the `toggled()` signal when checked on or off and can be connected to this signal to trigger an action.

An example of creating `QRadioButton` widgets can be seen in Listing 6-1.

## The `QGroupBox` Class

The `QGroupBox` widget provides a container for grouping other widgets with similar purposes together. A group box has a border with a title on the top. The title can also be checkable so that the child widgets inside the group box can be enabled or disabled when the checkbox is checked or unchecked.

A group box object can contain any kind of widget. Since `QGroupBox` does not automatically lay out its child widgets, you will need to apply a layout such as `QHBoxLayout` or `QGridLayout`. The following snippet of code demonstrates how to create a `QGroupBox` widget, add two radio buttons, and apply a layout:

```
effects_gb = QGroupBox("Effects") # The title can either be set in the constructor or with the setTitle
# Create instances of radio buttons
effect1_rb = QRadioButton("Strikethrough")
effect2_rb = QRadioButton("Outline")
# Set up layout and add child widgets to the layout
h_box = QHBoxLayout()
h_box.addWidget(effect1_rb)
h_box.addWidget(effect2_rb)
# Set the layout of the group box
effects_gb.setLayout(h_box)
```

For an example of another type of container in PyQt, check out the `QFrame` class in Chapter 7.

## The `QTabWidget` Class

Sometimes you may need to organize related information onto separate pages rather than create a cluttered GUI. The `QTabWidget` class provides a tab bar (created from the `QTabBar` class) with an area under each tab (referred to as a page) to present information and widgets related to each tab. Only one page is displayed at a time, and the user can view a different page by clicking the tab or by using a shortcut (if one is set for the tab).

There are a few different ways to interact with and keep track of the different tabs. For example, if the user switches to a different tab, the `currentChanged()` signal is emitted. You can also keep track of a current page's index with `currentIndex()`, or the widget of the current page with `currentWidget()`. A tab can also be enabled or disabled with the `setTabEnabled()` method.

**Tip** If you want to create an interface with multiple pages, but without the tab bar, then you should consider using a `QStackedWidget`. However, if you do use `QStackedWidget`, then you will need to provide some other means to switch between the windows, such as a `QComboBox` or a `QListWidget`, since there are no tabs.

The following example creates a simple application that includes `QRadioButton`, `QGroupBox`, and `QTabWidget` and a few other classes. The program shows how to set up a tab widget and organize the other widgets on the different pages.

```

# contact_form.py
# Import necessary modules
import sys
from PyQt5.QtWidgets import (QApplication, QWidget, QTabWidget, QLabel, QRadioButton, QGroupBox, QLineEdit)

class ContactForm(QWidget):
    def __init__(self):
        super().__init__()
        self.initializeUI()

    def initializeUI(self):
        """
        Initialize the window and display its contents to the screen.
        """

        self.setGeometry(100, 100, 400, 300)
        self.setWindowTitle('Contact Form Example')
        self.setupTabs()
        self.show()

    def setupTabs(self):
        """
        Set up tab bar and different tab widgets. Each tab is a QWidget that serves as a container for
        """

        # Create tab bar and different tabs
        self.tab_bar = QTabWidget(self)
        self.prof_details_tab = QWidget()
        self.background_tab = QWidget()
        self.tab_bar.addTab(self.prof_details_tab, "Profile Details")
        self.tab_bar.addTab(self.background_tab, "Background")
        # Call methods that contain the widgets for each tab
        self.profileDetailsTab()
        self.backgroundTab()

        # Create layout for main window
        main_h_box = QHBoxLayout()
        main_h_box.addWidget(self.tab_bar)
        # Set main window's layout
        self.setLayout(main_h_box)

    def profileDetailsTab(self):
        """
        Create the profile tab. Allows the user enter their name,
        address and select their gender.
        """

        # Set up labels and line edit widgets
        name_label = QLabel("Name")
        name_entry = QLineEdit()
        address_label = QLabel("Address")
        address_entry = QLineEdit()
        # Create group box to contain radio buttons
        sex_gb = QGroupBox("Sex")
        male_rb = QRadioButton("Male")
        female_rb = QRadioButton("Female")
        # Create and set layout for sex_gb widget
        sex_h_box = QHBoxLayout()
        sex_h_box.addWidget(male_rb)
        sex_h_box.addWidget(female_rb)
        sex_gb.setLayout(sex_h_box)
        # Add all widgets to the profile details page layout
        tab_v_box = QVBoxLayout()
        tab_v_box.addWidget(name_label)
        tab_v_box.addWidget(name_entry)
        tab_v_box.addWidget(address_label)
        tab_v_box.addWidget(address_entry)
        tab_v_box.addWidget(sex_gb)
        tab_v_box.addStretch()

```



```

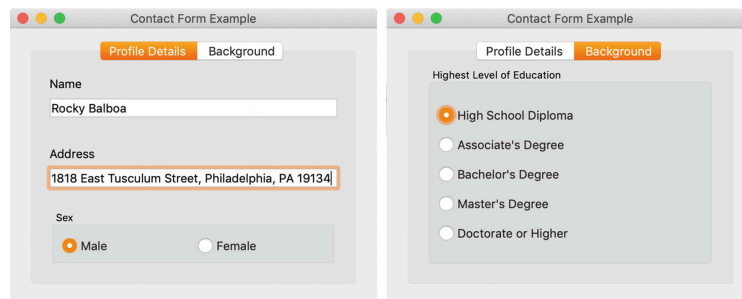
tab_v_box.addWidget(address_label)
tab_v_box.addWidget(address_entry)
tab_v_box.addStretch()

tab_v_box.addWidget(sex_gb)
# Set layout for profile details tab
self.prof_details_tab.setLayout(tab_v_box)
def backgroundTab(self):
    """
    Create the background tab. The user can select a
    """
    # Set up group box to hold radio buttons
    self.education_gb = QGroupBox("Highest Level of Education")
    # Layout for education_gb
    ed_v_box = QVBoxLayout()
    # Create and add radio buttons to ed_v_box
    education_list = ["High School Diploma", "Associate's Degree", "Bachelor's Degree", "Master's D
    for ed in education_list:
        self.education_rb = QRadioButton(ed)
        ed_v_box.addWidget(self.education_rb)
    # Set layout for group box
    self.education_gb.setLayout(ed_v_box)
    # Create and set for background tab
    tab_v_box = QVBoxLayout()
    tab_v_box.addWidget(self.education_gb)
    self.background_tab.setLayout(tab_v_box)
if __name__ == '__main__':
    app = QApplication(sys.argv)
    window = ContactForm()
    sys.exit(app.exec_())

```

**Listing 6-1** Example that shows how to use QTabWidget, QRadioButton, and QGroupBox classes

Figure 6-4 shows you how the GUI should look for each tab.



**Figure 6-4** The contact form GUI. The Profile Details tab (left) contains two labels and two line edit widgets as well as a group box with two radio buttons. The Background tab (right) consists of a single group box container with five radio buttons

## Explanation

Let's take a look at how to set up the tab widget and its child widgets in this example.

We begin by importing the necessary classes, including `QRadioButton`, `QTabWidget`, and `QGroupBox` from the `QtWidgets` module. Next, we set up the `ContactForm` class and initialize the window's size and title.

The next step is to set up the tab widget and each page in the `setupTabs()` method. The process to use `QTabWidget` is to first create an instance of the tab widget. Here we create `tab_bar`. Then, create a `QWidget` object for each page in the tab bar. There are two pages for this project, `profile_details_tab` and `background_tab`.

Insert the two pages into the tab widget using `addTab()`. Give each tab an appropriate label.

Finally, create the child widgets for each page and use layouts to arrange them. Two separate methods are created, `profileDetailsTab()` and `backgroundTab()`, to organize the two different pages. The labels and line edit widgets are set up like normal. For the `QRadioButton` objects, they are added to group boxes on their respective pages. In the `backgroundTab()` method, a `for` loop is used to instantiate each radio button and add it to the page's layout.

## Food Ordering GUI Solution

Now that we have taken a look at the new widgets in this chapter, we can finally move onto the code for the food ordering interface in Listing [6-2](#).

```
# food_order.py
# Import necessary modules
import sys
from PyQt5.QtWidgets import (QApplication, QWidget, QTabWidget, QLabel, QRadioButton, QButtonGroup, QGr
from PyQt5.QtGui import QPixmap
from PyQt5.QtCore import Qt
# Set up style sheet for the entire GUI
style_sheet = """
    QWidget{
        background-color: #C92108;
    }
    QWidget#Tabs{
        background-color: #FCEBCD;
        border-radius: 4px
    }
    QWidget#ImageBorder{
        background-color: #FCF9F3;
        border-width: 2px;
        border-style: solid;
        border-radius: 4px;
        border-color: #FABB4C
    }
    QWidget#Side{
        background-color: #EFD096;
        border-radius: 4px
    }
    QLabel{
        background-color: #EFD096;
        border-width: 2px;
        border-style: solid;
        border-radius: 4px;
        border-color: #EFD096
    }
}
```

```

QLabel#Header{
    background-color: #EFD096;
    border-width: 2px;
    border-style: solid;
    border-radius: 4px;
    border-color: #EFD096;
    padding-left: 10px;
    color: #961A07
}
QLabel#ImageInfo{
    background-color: #FCF9F3;
    border-radius: 4px
}
QGroupBox{
    background-color: #FCEBCD;
    color: #961A07
}
QRadioButton{
    background-color: #FCF9F3
}
QPushButton{
    background-color: #C92108;
    border-radius: 4px;
    padding: 6px;
    color: #FFFFFF
}
QPushButton:pressed{
    background-color: #C86354;
    border-radius: 4px;
    padding: 6px;
    color: #DFD8D7
}
"""
class FoodOrderGUI(QWidget):
    def __init__(self):
        super().__init__()
        self.initializeUI()
    def initializeUI(self):
        """
        Initialize the window and display its contents to the screen.
        """
        self.setMinimumSize(600, 700)

        self.setWindowTitle('6.1 - Food Order GUI')
        self.setupTabsAndLayout()
        self.show()
    def setupTabsAndLayout(self):
        """
        Set up tab bar and different tab widgets.
        Also, create the side widget to display items selected.
        """
        # Create tab bar, different tabs, and set object names
        self.tab_bar = QTabWidget(self)
        self.pizza_tab = QWidget()
        self.pizza_tab.setObjectName("Pizza")
        self.wings_tab = QWidget()
        self.wings_tab.setObjectName("Wings")
        self.tab_bar.addTab(self.pizza_tab, "Pizza")
        self.tab_bar.addTab(self.wings_tab, "Wings")
        # Call methods that contain the widgets for each tab

```

```

self.pizzaTab()
self.wingsTab()
# Set up side widget which is not part of the tab widget
self.side_widget = QWidget()
self.side_widget.setObjectName("Tabs")
order_label = QLabel("YOUR ORDER")
order_label.setObjectName("Header")
items_box = QWidget()
items_box.setObjectName("Side")
pizza_label = QLabel("Pizza Type: ")
self.display_pizza_label = QLabel("")
toppings_label = QLabel("Toppings: ")
self.display_toppings_label = QLabel("")
extra_label = QLabel("Extra: ")
self.display_wings_label = QLabel("")
# Set grid layout for objects in side widget

items_grid = QGridLayout()
items_grid.addWidget(pizza_label, 0, 0, Qt.AlignRight)
items_grid.addWidget(self.display_pizza_label, 0, 1)
items_grid.addWidget(toppings_label, 1, 0, Qt.AlignRight)
items_grid.addWidget(self.display_toppings_label, 1, 1)
items_grid.addWidget(extra_label, 2, 0, Qt.AlignRight)
items_grid.addWidget(self.display_wings_label, 2, 1)
items_box.setLayout(items_grid)
# Set main layout for side widget
side_v_box = QVBoxLayout()
side_v_box.addWidget(order_label)
side_v_box.addWidget(items_box)
side_v_box.addStretch()
self.side_widget.setLayout(side_v_box)
# Add widgets to main window and set layout
main_h_box = QHBoxLayout()
main_h_box.addWidget(self.tab_bar)
main_h_box.addWidget(self.side_widget)
self.setLayout(main_h_box)

def pizzaTab(self):
    """
    Create the pizza tab. Allows the user to select the type of pizza and topping using radio butto
    """
    # Set up widgets and layouts to display information

    # to the user about the page
    tab_pizza_label = QLabel("BUILD YOUR OWN PIZZA")
    tab_pizza_label.setObjectName("Header")
    description_box = QWidget()
    description_box.setObjectName("ImageBorder")
    pizza_image_path = "images/pizza.png"
    pizza_image = self.loadImage(pizza_image_path)
    pizza_desc = QLabel()
    pizza_desc.setObjectName("ImageInfo")
    pizza_desc.setText("Build a custom pizza for you. Start with your favorite crust and add any to
    pizza_desc.setWordWrap(True)
    h_box = QHBoxLayout()
    h_box.addWidget(pizza_image)
    h_box.addWidget(pizza_desc)
    description_box.setLayout(h_box)
    # Create group box that will contain crust choices
    crust_gbox = QGroupBox()

```

```

crust_gbox.setTitle("CHOOSE YOUR CRUST")
# The group box is used to group the widgets together,
# while the button group is used to get information
# about which radio button is checked
self.crust_group = QButtonGroup()
gb_v_box = QVBoxLayout()
crust_list = ["Hand-Tossed", "Flat", "Stuffed"]
# Create radio buttons for the different crusts and
# add to layout
for cr in crust_list:
    crust_rb = QRadioButton(cr)
    gb_v_box.addWidget(crust_rb)
    self.crust_group.addButton(crust_rb)
crust_gbox.setLayout(gb_v_box)
# Create group box that will contain toppings choices
toppings_gbox = QGroupBox()
toppings_gbox.setTitle("CHOOSE YOUR TOPPINGS")
# Set up button group for toppings radio buttons
self.toppings_group = QButtonGroup()
gb_v_box = QVBoxLayout()
toppings_list = ["Pepperoni", "Sausage", "Bacon", "Canadian Bacon", "Beef", "Pineapple", "Mushr"]
# Create radio buttons for the different toppings

and

# add to layout
for top in toppings_list:
    toppings_rb = QRadioButton(top)
    gb_v_box.addWidget(toppings_rb)
    self.toppings_group.addButton(toppings_rb)
self.toppings_group.setExclusive(False)
toppings_gbox.setLayout(gb_v_box)
# Create button to add information to side widget
# when clicked
add_to_order_button1 = QPushButton("Add To Order")
add_to_order_button1.clicked.connect(self.displayPizzaInOrder)
# Create layout for pizza tab (page 1)
page1_v_box = QVBoxLayout()
page1_v_box.addWidget(tab_pizza_label)
page1_v_box.addWidget(description_box)
page1_v_box.addWidget(crust_gbox)
page1_v_box.addWidget(toppings_gbox)
page1_v_box.addStretch()
page1_v_box.addWidget(add_to_order_button1, alignment=Qt.AlignRight)
self.pizza_tab.setLayout(page1_v_box)
def wingsTab(self):
    # Set up widgets and layouts to display information
    # to the user about the page
    tab_wings_label = QLabel("TRY OUR AMAZING WINGS")
    tab_wings_label.setObjectName("Header")
    description_box = QWidget()
    description_box.setObjectName("ImageBorder")
    wings_image_path = "images/wings.png"
    wings_image = self.loadImage(wings_image_path)
    wings_desc = QLabel()
    wings_desc.setObjectName("ImageInfo")
    wings_desc.setText("6 pieces of rich-tasting, white meat chicken that will have you coming back")
    wings_desc.setWordWrap(True)
    h_box = QHBoxLayout()
    h_box.addWidget(wings_image)
    h_box.addWidget(wings_desc)

```

```

description_box.setLayout(h_box)

wings_gbox = QGroupBox()
wings_gbox.setTitle("CHOOSE YOUR FLAVOR")
self.wings_group = QButtonGroup()
gb_v_box = QVBoxLayout()
wings_list = ["Buffalo", "Sweet-Sour", "Teriyaki", "Barbecue"]
# Create radio buttons for the different flavors and
# add to layout
for fl in wings_list:
    flavor_rb = QRadioButton(fl)
    gb_v_box.addWidget(flavor_rb)
    self.wings_group.addButton(flavor_rb)
wings_gbox.setLayout(gb_v_box)
# Create button to add information to side widget
# when clicked
add_to_order_button2 = QPushButton("Add To Order")
add_to_order_button2.clicked.connect(self.displayWingsInOrder)
# Create layout for wings tab (page 2)
page2_v_box = QVBoxLayout()
page2_v_box.addWidget(tab_wings_label)
page2_v_box.addWidget(description_box)
page2_v_box.addWidget(wings_gbox)
page2_v_box.addWidget(add_to_order_button2, alignment=Qt.AlignRight)
page2_v_box.addStretch()
self.wings_tab.setLayout(page2_v_box)
def loadImage(self, img_path):
    """
    Load and scale images

    """
    try:
        with open(img_path):
            image = QLabel(self)
            image.setObjectName("ImageInfo")
            pixmap = QPixmap(img_path)
            image.setPixmap(pixmap.scaled(image.size(), Qt.KeepAspectRatioByExpanding, Qt.SmoothTra
            return image
    except FileNotFoundError:
        print("Image not found.")
def collectToppingsInList(self):
    """
    Create list of all checked toppings radio buttons.
    """
    toppings_list = [button.text() for i, button in enumerate(self.toppings_group.buttons()) if but
    return toppings_list
def displayPizzaInOrder(self):
    """
    Collect the text from the radio buttons that are checked on pizza page. Display text in side wi
    checkedButton() returns the buttons that are checked in the QButtonGroup.
    """
    try:
        pizza_text = self.crust_group.checkedButton().text()
        self.display_pizza_label.setText(pizza_text)
        toppings = self.collectToppingsInList()
        toppings_str = '\n'.join(toppings)
        self.display_toppings_label.setText(toppings_str)
        self.repaint()

```

```

except AttributeError:
    print("No value selected.")
    pass
def displayWingsInOrder(self):
    """
    Collect the text from the radio buttons that are checked on wings page. Display text in side wi
    """
    try:
        text = self.wings_group.checkedButton().text() + " Wings"
        self.display_wings_label.setText(text)
        self.repaint()
    except AttributeError:
        print("No value selected.")
        pass
if __name__ == '__main__':
    app = QApplication(sys.argv)
    app.setStyleSheet(style_sheet)
    window = FoodOrderGUI()
    sys.exit(app.exec_())

```

*Listing 6-2* Code for food ordering GUI

When finished, your GUI should look similar to the one in Figure [6-2](#).

### Explanation

Let's first import the modules we need for this project. Next, the properties for the widgets in this application are prepared in the `style_sheet` variable. We will get to how this works shortly.

Create the structure for the tabs and layout for the main window in `setUpTabsAndLayout()`. Set up instances of the `QTabWidget` and `QWidget` objects that will be used for the pages of the tabs. The two tabs are the `pizza_tab`, to display choices for building your own pizza, and the `wings_tab`, to show choices for wings flavors.

Some of the widgets in this GUI are given an ID selector using the `setObjectName()` method. For example, `pizza_tab` is given the Tabs ID selector. This name will be used in the `style_sheet` to differentiate this widget from other `QWidget` objects with a different style.

```

self.pizza_tab = QWidget()
self.pizza_tab.setObjectName("Tabs")

```

The `side_widget` is used to give feedback to users of their choices and can be seen even if the user switches tabs. All of the child widgets for `side_widget` are then arranged in a nested layout and added to the main `QHBoxLayout`.

The `pizzaTab()` method creates and arranges the child widgets for the first tab, `pizza_tab`. The top of the first page gives users information about the purpose of the tab using images and text. The `wingsTab()` method is set up in a similar manner.

`QRadioButton` widgets are grouped together using group boxes. This allows each group to have a title. The `QGroupBox` class does provide exclusivity to radio buttons, but to get the type of functionality to find out which buttons are checked

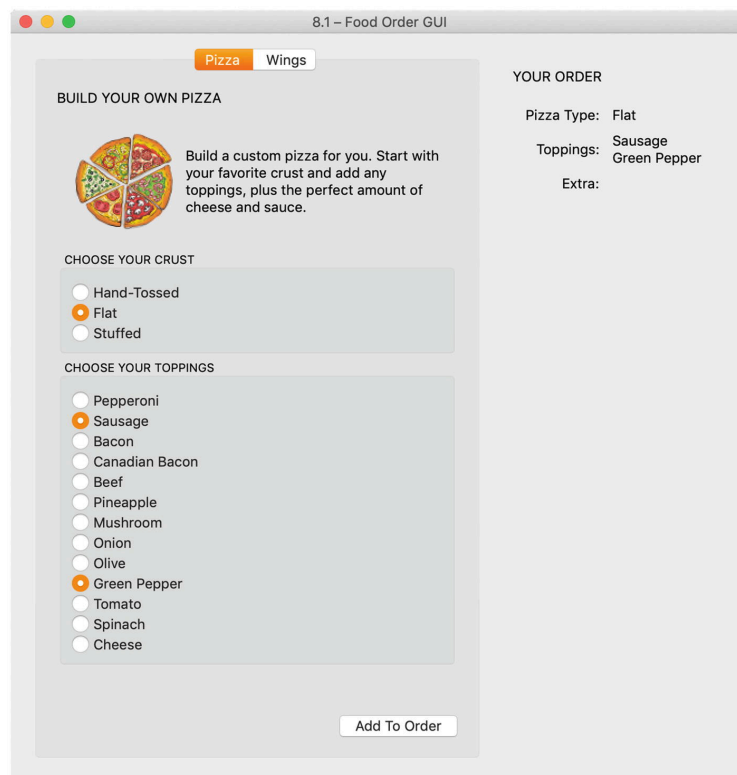
and return their text values, the `QRadioButton` objects are also grouped using `QGroupButton`. Refer to Chapter 4 for more information about `QButtonGroup`. While only one radio button can be selected in the `Crust` group, users need to be able to select more than one topping. This is achieved by setting the exclusivity of the `toppings_group` to `False`.

```
self.toppings_group.setExclusive(False)
```

If users press the `add_to_order_button` on either page, the text from the selected radio buttons is displayed in the `side_widget`. A Python `try-except` clause is used to ensure that the user has selected radio buttons.

### Applying the Style Sheet

If a style sheet is not applied to the food ordering GUI, then it will use your system's native settings to style the application. Figure 6-5 shows what this looks like on MacOS.



**Figure 6-5** The food ordering GUI before the style sheet is applied

In the beginning of the program, you will notice the `style_sheet` variable that holds all of the different style specifications for the different widgets.

To apply a general style to all widgets of one type, you only need to specify the class. For example, the following code gives all `QWidget` objects a red background:

```
QWidget{
    background-color: #C92108;
}
```



But if a `QWidget` object has a specified ID selector such as `Tabs`, then it will get a tan background and rounded corners.

```
QWidget#Tabs{
    background-color: #FCEBCD;
    border-radius: 4px
}
```

Other widget's properties are set up in a similar manner. The style sheet is applied to the entire application by calling `setStyleSheet()` on the `QApplication` object.

```
app.setStyleSheet(style_sheet)
```

The final GUI with customized colors, borders, and fonts can be seen in Figure [6-2](#).

## Event Handling in PyQt

The concept of signals and slots in PyQt was briefly introduced in Chapter [3](#). Event handling in PyQt uses signals and slots to communicate between objects. Signals are typically generated by a user's actions, and slots are methods that are executed in response to the signal. For example, when a `QPushButton` is pushed, it emits a `clicked()` signal. This signal could be connected to the PyQt slot `close()` so that a user can quit the application when the button is pressed.

The `clicked()` signal is but one of many predefined Qt signals. The type of signals that can be emitted differs according to the widget class. PyQt delivers events to widgets by calling specific, predefined handler functions. These can range from functions related to window operations, such as `show()` or `close()`, to GUI appearances with `setStyleSheet()`, to mouse press and release events, and more.

The way in which event handlers deal with events can also be reimplemented. You saw an example of this back in Chapter [3](#) when the `closeEvent()` function was modified to display dialog boxes before closing the application.

The following example, Listing [6-3](#), shows a very simple example of how to reimplement the `keyPressEvent()` function.

```
# close_event.py
# Import necessary modules
import sys
from PyQt5.QtWidgets import QApplication, QMainWindow
from PyQt5.QtCore import Qt
class Example(QMainWindow):
    def __init__(self):
        super().__init__()
        self.initializeUI()
    def initializeUI(self):
        """
        Initialize the window and display its contents to the screen.
```

```

"""
    self.setGeometry(100, 100, 300, 200)
    self.setWindowTitle('Event Handling Example')
    self.show()
    def keyPressEvent(self, event):
        if event.key() == Qt.Key_Escape:
            print("Application closed.")
            self.close()
if __name__ == '__main__':
    app = QApplication(sys.argv)
    window = Example()
    sys.exit(app.exec_())

```

*Listing 6-3* Code to demonstrate how to modify event handlers

## Explanation

Whenever a user presses a key on the keyboard, it sends a signal to the computer. If you want to give certain keys abilities, then you will need to use the `keyPressEvent()`.

The `keyPressEvent()` function checks for events, which in this case are the signals being sent from keys. If the key pressed is the `Escape` key, then the application calls the `close()` function to quit the application.

Of course, you can check for any type of key with the `keyPressEvent()` and cause it to perform any number of actions.

## Creating Custom Signals

We have taken a look at some of PyQt's predefined signals and slots. For many of the projects in previous chapters, we have also created custom slots to handle the signals emitted from widgets.

Now let's see how we can create a custom signal using `pyqtSignal` to change a widget's style sheet in Listing [6-4](#).

```

# color_event.py
# Import necessary modules
import sys
from PyQt5.QtWidgets import QApplication, QMainWindow, QLabel
from PyQt5.QtCore import Qt, pyqtSignal, QObject
class SendSignal(QObject):
    """
    Define a signal change_style that takes no arguments.
    """
    change_style = pyqtSignal()
class Example(QMainWindow):
    def __init__(self):
        super().__init__()
        self.initializeUI()
    def initializeUI(self):
        """
        Initialize the window and display its contents to the screen.
        """
        self.setGeometry(100, 100, 300, 200)
        self.setWindowTitle('Create Custom Signals')
        self.setupLabel()
        self.show()

```

```

def setupLabel(self):
    """
    Create label and connect custom signal to slot.
    """
    self.index = 0 # index of items in list
    self.direction = ""
    self.colors_list = ["red", "orange", "yellow", "green", "blue", "purple"]
    self.label = QLabel()
    self.label.setStyleSheet("background-color: {}".format(self.colors_list[self.index]))
    self.setCentralWidget(self.label)
    # Create instance of SendSignal class, and
    # connect change_style signal to a slot.
    self.sig = SendSignal()
    self.sig.change_style.connect(self.changeBackground)
def keyPressEvent(self, event):
    """
    Reimplement how the key press event is handled.
    """
    if (event.key() == Qt.Key_Up):
        self.direction = "up"
        self.sig.change_style.emit()
    elif event.key() == Qt.Key_Down:
        self.direction = "down"
        self.sig.change_style.emit()
def changeBackground(self):
    """
    Change the background of the label widget when a keyPressEvent signal is emitted.
    """
    if self.direction == "up" and self.index < len(self.colors_list) - 1:
        self.index = self.index + 1
        self.label.setStyleSheet("background-color: {}".format(self.colors_list[self.index]))
    elif self.direction == "down" and self.index > 0:
        self.index = self.index - 1
        self.label.setStyleSheet("background-color: {}".format(self.colors_list[self.index]))
if __name__ == '__main__':
    app = QApplication(sys.argv)
    window = Example()
    sys.exit(app.exec_())

```

**Listing 6-4** Creating a custom signal to change the background color of a QLabel widget

This example creates a simple GUI with a QLabel widget as the central widget of the main window.

## Explanation

The `pyqtSignal` factory and `QObject` classes are imported from the `QtCore` module. The `QtCore` module and `QObject` classes provide the mechanics for signals and slots.

The `SendSignal` class creates a new signal called `change_style` from the `pyqtSignal` factory. This signal will be generated whenever the user presses either the up arrow key or the down arrow key. By pressing up or down, the user can change the background color of the `QLabel` object.

```
self.sig.change_style.connect(self.changeBackground)
```

When the user presses `Key_Up`, `direction` is set equal to "up", and a `change_style` signal is emitted.

```
self.sig.change_style.emit()
```

This signal is connected to the `changeBackground()` slot which updates the color of the label by calling the `setStyleSheet()` method.

It works in a similar fashion when the down key is pressed.

## Summary

In this chapter, we saw how to use Qt Style Sheets to modify the appearance of widgets to better fit the purpose and look of an application. Applying customizations in a consistent and attentive manner can greatly influence the usability of a user interface.

Allowing the user to also have some control over the look of the window can improve the user's experience. This can be done in a number of ways – through the menu or toolbar, using a context menu, or even through simple presses of keys on the keyboard.

Chapter 7 will introduce Qt Designer, a tool that will make the process for designing GUIs much simpler.